# INF1103 Report: Error Logging

*Team ID:* Lab-P2-G2

*Team name:* JSON Derulo

*Team members:*


| Team Member | Student ID | Email |
|---|---|---|
| Ang Jia Yi | 2603644 | 2603644@sit.singaporetech.edu.sg |
| Muhamed Naim Bin Zakir | 2602953 | 2602953@sit.singaporetech.edu.sg |
| Ng Jing Jie | 2604438 | 2604438@sit.singaporetech.edu.sg |
| Park Su Bin | 2604051 | 2604051@sit.singaporetech.edu.sg |
| Tan Hwee Ning Zoey | 2605299 | 2605299@sit.singaporetech.edu.sg |
| Tan Soon Meng Owen | 2601837 | 2601837@sit.singaporetech.edu.sg |


Date of submission: 14/10/2026

# Introduction:

In daily operations, companies often generate large volumes of error logs from their systems and applications. These logs contain valuable information about system failures, performance issues, and potential disruptions that require investigation. For example, a healthcare organisation may encounter network packet loss that degrades vital signs monitoring or failed DICOM image ingestion caused by incomplete image payloads.

As the volume of logs increases, manually reviewing and identifying individual issues becomes time-consuming and inefficient. Furthermore, similar errors may appear across multiple systems, making it difficult to identify recurring problems and understand the overall operational status.

To address this challenge, an automated approach is needed to process error logs, categorise the different issues, and consolidate the results into a structured report. This report aims to explore how error logs can be systematically analysed to identify distinct issues and present the findings in a clear and organised format, enabling teams to investigate problems more efficiently and make informed decisions.

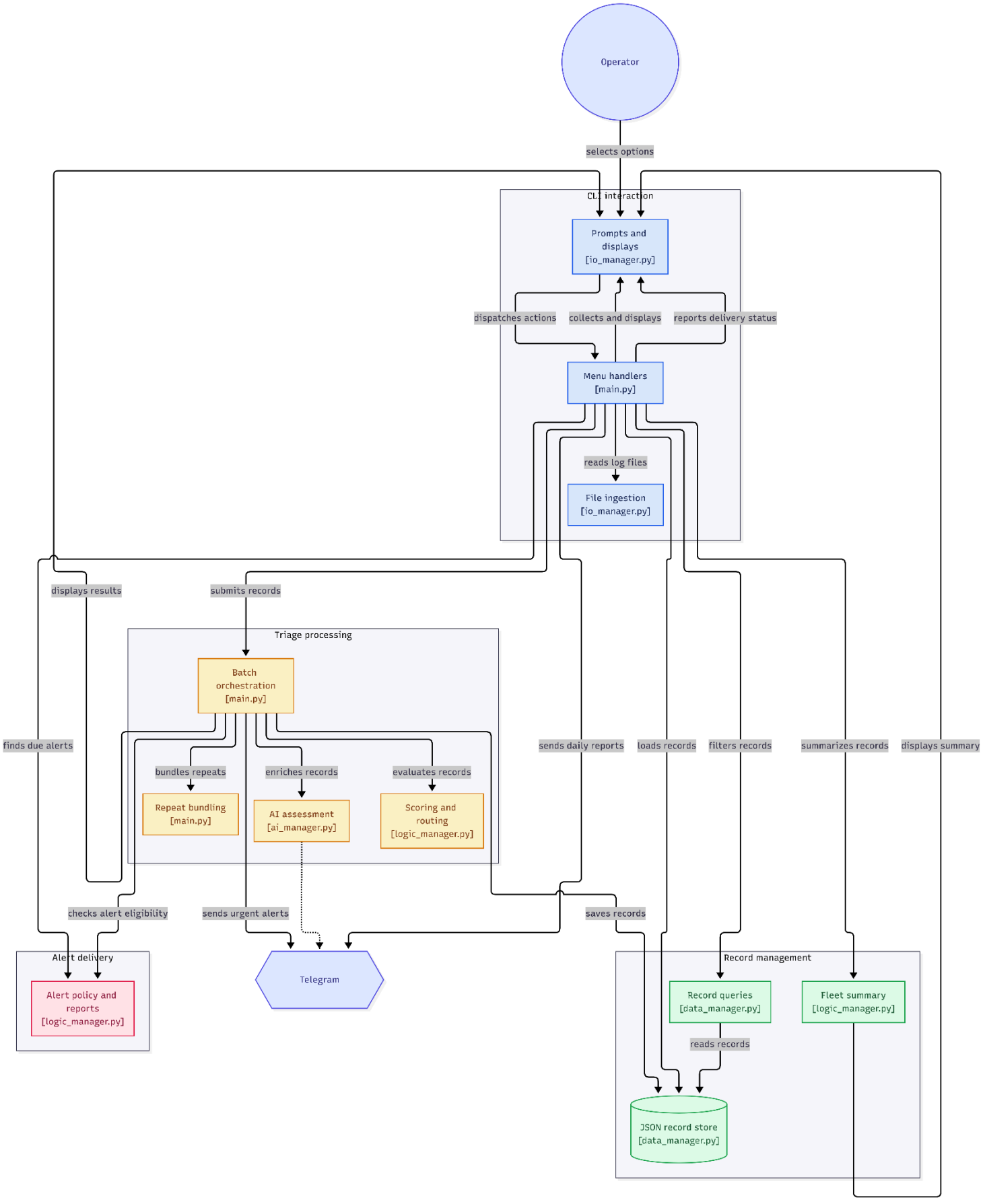
# Data Structure and Storage Design:

The application is developed using Python and follows a modular design approach. The system is divided into four main components: the Logic Manager, AI Manager, Data Manager, and Input/Output Manager. Each manager is responsible for a specific set of functions, helping to separate the system's responsibilities and improve code organisation, maintainability, and readability.

The application uses JSON files as its primary data storage format. These files allow information to be retained between program executions. JSON was chosen because it is human-readable, easy to modify, and capable of storing structured data without requiring a separate database management system.

The stored data is organised as a JSON array containing machine error log records, with each element representing an individual log. Each record consists of three main sections: general error information, AI analysis, and decision details. The general information contains machine identifiers, error details, timestamps, and reporter information for traceability. The ai object stores AI-generated analysis, including the potential root cause, severity, and recommended actions. Lastly, the decision object records the system's handling decisions, including notification status, processing route, and whether human review is required.

# Data Flow Diagram:



# Error Exception Handling Matrix

| Exception Scenario | Possible Causes | Detection Method | Handling Strategy | User Outcome |
|---|---|---|---|---|
| AI and API errors | Missing API key | Validates response/Check environment | Records error without assessment | Routes to Human Review |
| | Authentication failure | | | |
| | Connection timeouts | ai_manager.py catches requests.RequestException | Retries up to 3 times with logging | Returns None; AI assessment unavailable |
| | HTTP failures | | | |
| Storage & JSON | Missing files | data_manager.py handles missing logs | Returns an empty list | Starts with fresh log structure |
| | FileNotFoundError | | | |
| | Malformed JSON syntax | Schema and type validation checks | Rejects invalid entries during load | Skips corrupt records; continues execution |
| | Non-dictionary records | | | |
| Concurrency | Concurrent file access collision | Thread/process lock implementation | data_manager.py wraps operations in a lock | Prevents write corruption safely |
| User input | EOF/interrupt | io_manager.py catches EOFError, KeyboardInterrupt | Cancel the action cleanly | Returns None |
| | Closed Stream | | | |
| | Invalid choices | io_manager.py validation checks | Catch ValueError prompt retry | Prompts user to input valid data |
| | Missing Values | | | |
| | Bad timestamps | | | |
| Log-file errors | Malformed fields | Field length & type validation checks | Validate schema before processing record | Skips invalid line; continues batch |
| | Invalid formatting | | | |
| | Read permission errors | io_manager.py catches OSError | Log error internally; bypass raise | Returns error message fallback |
| | Storage failures | | | |

| Exception Scenario | Possible Causes | Detection Method | Handling Strategy | User Outcome |
|---|---|---|---|---|
| | Write failure | io_manager.py catches IOError/OSError | Catch exception; log event | Returns error message fallback |
| Rules and Telegram delivery | Missing configuration | logic_manager.py catches requests.RequestException | Log warning; report Telegram unreachable | Skips alert; logs event locally |
| | Connection drops | | | |
| | HTTP errors | logic_manager.py evaluates API response code | Log status code and failure notice | Bypasses notification fallback |